

Predicting Album Scores Based on Their Reviews with Sentiment Analysis*

Jason Serduke¹

¹ California State University, East Bay, Hayward, United States

Abstract

The problem was how I could best sort music reviews into the positive and negative ones, comparing the trade-offs of a few different methods. Particularly, I wanted to see how naive approaches, i.e., what would've been my go-to strategies before this class, compared to a model that I trained specifically for the purpose.

My baseline methods for assessment were having a human sort through the reviews manually, and feeding them to an out-of-the-box large language model, in this case ChatGPT, to make predictions on. For my personalized model, I chose a recurrent neural network, specifically with a long short-term memory network and a bidirectional architecture.

First, I collected the data by scraping reviews and their corresponding scores from a music review website. Next, I trained models of varying architectures with different layers to see which had the most potential. Then, I fine-tuned additional hyperparameters of the model. Finally, I assessed the final model's performance.

In the end, the recurrent neural network did not come very close to the performance of the LLM, but did nearly as well as the human being, expending much less time in its assessments. While this is offset by the effort taken to actually train the model, on a larger scale of predictions over a longer period of time, it could become the more efficient approach. Additionally, it does not require paying for an AI subscription, which may, now or in the future, not be affordable for operations of all sizes.

1. Introduction

I knew beforehand, even before the bounds in improvement from modern large language models, that neural networks could categorize samples into different categories, whether they came in the form of text, images, or audio. This type of natural language processing particularly interests me because, even if it's not perfect, it can help people save a lot of time with these classification tasks that, when performed manually, can often be relatively te-

dious rather than rewarding.

As far as specific projects go, I took inspiration and learned a lot from "Text Classification with an RNN", which follows a tutorial categorizing film reviews [7]. I wanted to take what I learned from it, additional tutorials, and class lectures and assignments to see if I could apply it to my own area of interest in music.

In order to make this work, I had to collect many samples of album reviews and scores from a website. Then, I had to decide on what specific architecture I would employ for my task and train the model, frequently tuning hyperparameters and reassessing accuracy to maximize the model's performance. I also needed something to compare the model's results against, since I'm not familiar with any other work for this particular application, so I tested its accuracy against both a human and an LLM.

The results were that the LLM performed far ahead of the other two in accuracy, and my model performed just a bit worse than the human. While the LLM and RNN were both free, this is because I had a free subscription to ChatGPT and Google Colab credits. If these costs weren't already covered, the LLM approach would cost a lot more than the RNN, and the human would cost much more than both. In terms of time, the RNN made an inference the quickest, followed by the LLM, and then the human.

2. Technical approach

When I learned that a recurrent neural network could perform classification tasks like sentiment analysis on text data, it immediately became a compelling choice for my architecture. By feeding the previous hidden state back into the weight matrix with the current word, it has access to past context to make its determination [1]. An RNN can also be enhanced by making it bidirectional, which allows it to use context from the beginning to the current word, as well as from the end to the current word, capturing meaning before and after the current word [1]. Finally, long short-term memory networks, an extension of RNNs, improve upon the architecture by removing information that is no longer relevant or adding information that may be needed later [1]. It also helps to alleviate the vanishing gradients issue, in which many repeated multiplication op-

*<https://github.com/jserduke/review-predictions>

erations cause gradients to approach zero [1].

Using these concepts, I constructed my model with TensorFlow out of a linear stack of layers, Sequential [13]: TextVectorization, the preprocessing layer [12], an Embedding layer [10], the Bidirectional LSTM layer [8, 11], capped off with two Dense layers [9]. I also considered alternative approaches, like using a transformer or fine-tuning a pretrained large language model. While these would be completely valid, and may have had potential to perform better, the transformer architecture was already represented by one of my baselines, ChatGPT, and I just wanted to keep the focus of my project pretty tight, so I determined it to be beyond the current scope.

3. Experimental setup

3.1. Data

The data that I’m working with is a set of pairs, with the first piece in each being a written review, a sequence of text which may range from just a few characters to several pages in length, and the second piece being a score, an integer between 0 and 100, inclusive. I sourced the data myself by scraping it from the music review website, Album of The Year. The reviews were collected from a wide variety of albums released in the 2010s and 2020s. In the end, the total number of reviews collected came out to 68,688.

One issue that I ran into was that there were way more positive reviews than negative reviews, which risked giving the positive ones way more emphasis and “weight” in the model. In order to compensate, I went back to the website and specifically sought out more negative reviews. Afterward, the numbers of positive and negative reviews still weren’t even, but I felt like I finally had a great enough sample of the latter, so I randomly extracted an even number of positive reviews from the total dataset to create the final dataset that I would be using, consisting of an equal number of positive and negative reviews, coming out to 15,458.

Another choice that I had to make regarding the data was what scores exactly should be considered positive versus negative. It didn’t feel like there was one objectively correct answer, but looking over some of the reviews and their corresponding scores, it was clear to me that sentiment wasn’t evenly distributed over the possible range of values. For example, in most cases, a review connected to a score of 50 didn’t come off as neutral (as one might expect), but rather negative, and a 70, which one might expect to be pretty positive, felt neutral, or perhaps just slightly positive. To account for this, scores between 0 and 49, inclusive, were considered negative, and scores from 80 to 100, inclusive, were considered positive.

Beyond filtering out the neutral reviews, those corresponding with scores between 50 and 79, inclusive, I chose

not to do any additional preprocessing. I made this decision because I wanted to be able to make an inference on as wide of a range of reviews as possible rather than having to exclude them from being able to work with the model. As a result, perhaps most notably, there are multiple languages included in the dataset, with the most common after English being Spanish. In addition to aligning closer with goals for use cases, I also hadn’t run across many examples where multiple languages were included in the dataset, so I was interested to see if it could still work or if not excluding them would have deleterious effects on model performance.

Table 1. Size of sample for each subset.

Subset	Percent	Sample size
Training	64	9893
Validation	16	2473
Test	20	3092
Total	100	15,458

3.2. Experiments

3.2.1. Recurrent neural network

In order to maximize the performance of the model, I tried a lot of different options with the particular structure of the recurrent neural network and hyperparameters.

The first thing that I tested out was whether performance could be improved by stacking two long short-term memory layers on top of each other in the model. I thought that this would be promising but it actually didn’t improve performance at all and training a model with multiple LSTM layers significantly increased the processing time.

Another aspect that I tested out was the optimal size of the vocabulary. I had read that making the vocabulary too big could slow training, so I started it out pretty low and doubled it upon improvement. Eventually, I uncapped the vocabulary and it didn’t turn out to noticeably slow the training process. Overall, I consider increasing the vocabulary size to have made one of the greatest differences in terms of performance of the model.

When it was time to test out different sets of hyperparameters, I first started out just manually passing in different combinations and simply using my intuition to infer what might be working. However, this was quite time-consuming, so I explored my options for an automated hyperparameter tuner. I settled on KerasTuner implementing the Hyperband algorithm, which tests out different sets of hyperparameters, keeps track of which ones perform the best, and can also cut the training off early before the full number of epochs if there is reasonable confidence that it won’t surpass the best model so far [2, 3, 5].

Table 2. Hyperparameter options.

Hyperparameter	Options
Embedding output dims	16, 32, 48, ..., 160
LSTM units	16, 32, 48, ..., 160
Dense units	16, 32, 48, ..., 160
Activation function #1	relu, sigmoid, softmax, tanh, linear
Activation function #2	relu, sigmoid, softmax, tanh, linear
Learning rate	1e-5, 2e-5, 5e-5, 1e-4, 2e-4, ..., 5e-3

3.2.2. Baselines

Finally, I needed baseline values to compare the results of the RNN against to actually understand whether it had performed well or not. For one, I recruited a volunteer to test out how well a human could categorize the reviews. I randomly selected and printed out two hundred album reviews, and asked the participant to make their best guess as to whether each corresponded to a high score or a low score. Afterward, I compared their predictions to the key and calculated the accuracy. For the second baseline method, I asked an LLM, ChatGPT, to make predictions on two hundred reviews. I used the following zero-shot prompt:

BACKGROUND: I scraped reviews and their corresponding scores from an album review website. I split the reviews into two groups: HIGH, with a score of 80 or greater, and LOW, with a score of 49 or less. Scores in between 50 and 79 have been excluded altogether.

INSTRUCTIONS: With the provided review, guess whether it corresponds to a HIGH score or a LOW score. Only respond with a single word: "HIGH" or "LOW".

RESTRICTION: Do not do any web searches.

REVIEW: [review]

SCORE:

I compiled all of the output from the prompts, graded them against the key, and then recorded its accuracy.

4. Results

After testing out many different combinations of hyperparameters, the optimal set that I found managed to reach an accuracy of 83.70% with a loss of 0.4060.

The recurrent neural network performed significantly worse than the large language model, but actually only a bit worse than the human.

The results surprised me in a few different manners. First, I was surprised that the RNN did nearly as well as the human participant, as beforehand, I had expected the latter to blow the former out of the water. However, I think that this makes sense when you consider that there were some

Table 3. Optimal hyperparameters found.

Hyperparameter	Optimal value
Text vectorization max tokens	None
Embedding output dims	48
LSTM units	144
Dense units	128
Activation function #1	tanh
Activation function #2	linear
Learning rate	0.005

Table 4. RNN performance against baselines.

Method	Accuracy
Recurrent neural network	0.8370
Human	0.8500
Large language model	0.9550

Spanish reviews that the participant had difficulty parsing. In addition, I noticed that, from my subjective judgment, the participant seemed to miss what I perceived to be some of the more straightforward classifications toward the end of the set. While this could be a coincidence, it may suggest that the participant experienced some fatigue that had an effect on their performance by the end, which the RNN is immune to.

Secondly, I expected ChatGPT to perform well, but I was surprised about just how well it did. I think that being trained to understand multiple languages boosted its performance, as there were some Spanish reviews in the set. Overall, though, I think that it makes sense that the LLM would perform better than the RNN, as it's been trained on so much more data and relies on a different underlying architecture, the transformer.

While comparing the accuracy of the methods is pretty simple, the metrics of time and cost are a bit more nebulous. The upfront time investment of training the RNN was much higher than creating an LLM prompt or the test for the participant. However, the human participant made their assessments the slowest, ChatGPT took up to a few seconds, and the RNN could make thousands of inferences in less than a minute. As a result, in the long run, over many predictions, the RNN could end up being more time efficient than the human and the LLM.

Similarly for cost, hiring a technician to train a recurrent neural network comes with a higher initial price tag than just pitching the reviews straight to a human or an LLM, but being able to run your own RNN afterward rather than paying the human or subscribing to ChatGPT could pay off in the long run.

5. Conclusions and future work

From this experiment, I learned that the LLM performs the best at categorizing reviews into those with high scores and those with low scores, and was significantly better than the other methods. Manual sorting by a human was only a bit better than the recurrent neural network. In the short term, the large language model was the best for time and cost, but in the long run, the recurrent neural network could be worth the upfront investment.

For future work, I believe that it would be worth it to take more precise measurements of the time and monetary costs of setting up and running each of the methods. In addition, I think that it would be really interesting to test these methods against other ones, like a transformer or a pretrained local LLM that could be fine-tuned for assessing reviews.

In a similar vein to this experiment, I think that making predictions on mobile app reviews could be really interesting. It could potentially also serve a purpose in funneling app users to either the app store upon detection of a positive review, or to an internal feedback system for a negative one.

6. Team work division and overall experience

As a solo project, I, Jason Serduke, wrote up the proposal, collected the data, presented the progress check, came up with the architecture, trained the model, tuned the hyperparameters, assessed the baseline methods, wrote up this final report, and will do the final presentation.

Acknowledgments

I would like to thank my mother, Cherise Serduke, for donating her time to be my human participant.

References

- [1] Jurafsky, D., & Martin, J. H. (2026). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with*

- language models* (3rd ed.). https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf
- [2] O'Malley, T., Bursztein, E., and Long, J., Chollet, F., and Jin, H., Invernizzi, L., & others (2019). *Hyperband Tuner*. https://keras.io/keras_tuner/api/tuners/hyperband/
- [3] O'Malley, T., Bursztein, E., and Long, J., Chollet, F., and Jin, H., Invernizzi, L., & others (2019). *KerasTuner*. https://keras.io/keras_tuner/
- [4] TensorFlow. (2024, August 31). *Basic text classification*. https://www.tensorflow.org/tutorials/keras/text_classification
- [5] TensorFlow. (2024, August 16). *Introduction to the Keras Tuner*. https://www.tensorflow.org/tutorials/keras/keras_tuner
- [6] TensorFlow. (2024, August 16). *Load text*. https://www.tensorflow.org/tutorials/load_data/text
- [7] TensorFlow. (2024, March 23). *Text classification with an RNN*. https://www.tensorflow.org/text/tutorials/text_classification_rnn
- [8] TensorFlow. (2024, June 7). *tf.keras.layers.Bidirectional*. https://www.tensorflow.org/api_docs/python/tf/keras/layers/Bidirectional
- [9] TensorFlow. (2024, June 7). *tf.keras.layers.Dense*. https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dense
- [10] TensorFlow. (2024, June 7). *tf.keras.layers.Embedding*. https://www.tensorflow.org/api_docs/python/tf/keras/layers/Embedding
- [11] TensorFlow. (2024, June 7). *tf.keras.layers.LSTM*. https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM
- [12] TensorFlow. (2024, June 7). *tf.keras.layers.TextVectorization*. https://www.tensorflow.org/api_docs/python/tf/keras/layers/TextVectorization
- [13] TensorFlow. (2024, June 7). *tf.keras.Sequential*. https://www.tensorflow.org/api_docs/python/tf/keras/Sequential

Address for correspondence:

Jason Serduke
25800 Carlos Bee Blvd., Hayward, CA 94542
jserduke@horizon.csueastbay.edu